

BÁO CÁO DỰ ÁN LẬP TRÌNH SOCKET TCP

Xây dựng Hệ thống Chat 1:1 & Truyền Tập dữ liệu Binary qua Giao thức TCP/IP

Học phần: Lập trình Mạng / Socket Lab	Môi trường thực thi: Python 3.14 (Windows)
Bài thực hành: Bài 1 (Chat 1:1) & Bài 2 (File Transfer)	Cổng kết nối (Ports): 5000 (Chat) 5001 (File)
Kiến trúc mạng: Client - Server (TCP/IP)	Thời gian hoàn thành: 24/09/2026

TÓM TẮT TỔNG QUAN DỰ ÁN: Dự án này đã hoàn thành nâng cấp 2 ứng dụng mạng cơ bản dựa trên bộ mã nguồn mẫu. Bài 1 đáp ứng đầy đủ 4 yêu cầu phát triển (REQ-1 Username handshake, REQ-2 Lệnh tự động /time, REQ-3 Lệnh tự động /whoami, REQ-4 Khả năng chống crash & ngăn tin nhắn rỗng). Bài 2 đáp ứng chuẩn giao thức ứng dụng 3 bước truyền dữ liệu byte stream hỗ trợ đa dạng định dạng tệp (văn bản, ảnh PNG/JPG, PDF, ZIP) với cơ chế xác nhận OK/ERROR và dọn dẹp tệp lỗi.

1. Nguyên lý Kỹ thuật & Kiến trúc Tổng quan

Giao thức TCP (Transmission Control Protocol) là giao thức truyền tải hướng kết nối (connection-oriented), đảm bảo tính tin cậy, đúng thứ tự và không mất mát dữ liệu giữa hai điểm cuối trên mạng IP. Trong bài lab này, thư viện chuẩn socket của Python được sử dụng để xây dựng các kênh truyền TCP luồng (SOCK_STREAM).

Tiêu chí Kỹ thuật	Bài 1: Chat TCP 1:1	Bài 2: File Transfer TCP
Địa chỉ Lắng nghe (Server)	0.0.0.0:5000	0.0.0.0:5001
Chế độ làm việc	Kênh hội thoại luân phiên / Tự động	Truyền Byte Stream từng khối (Chunked)
Định dạng Giao thức	UTF-8 kết thúc bằng ký tự \n	Header UTF-8 + Data Payload (Bytes)
Cơ chế Khôi phục Lỗi	Bắt lỗi Socket, giữ Server luôn alive	Xóa tệp lỗi dở dang khi bị ngắt kết nối

2. Bài 1: Chi tiết Triển khai Chat TCP 1:1

Mục tiêu bài này là duy trì sự đơn giản của mã nguồn gốc đồng thời bổ sung đầy đủ 4 yêu cầu phát triển kỹ thuật:

Mã Yêu cầu	Mô tả Tính năng	Giải pháp Kỹ thuật Triển khai
REQ-1 Username Handshake	Client nhập Username ngay sau khi kết nối. Server hiển thị thông báo với định dạng: <tên> từ <IP:port> đã kết nối	Ngay khi server.accept() thành công, luồng xử lý gọi recv_line() để nhận tên Client trước khi đi vào vòng lặp chat chính.

REQ-2 Lệnh /time	Khi client gửi /time, server tự động phản hồi lại thời gian hiện tại mà không cần người dùng ở server gõ tay.	Server bắt điều kiện <code>cmd.lower() == '/time'</code> , sử dụng <code>datetime.now()</code> lấy chuỗi ngày giờ và trả về phản hồi tự động ngay lập tức.
REQ-3 Lệnh /whoami	Server trả lại địa chỉ IP và source port mà server nhìn thấy từ client.	Server trích xuất bộ giá trị <code>client_address[0]</code> (IP) và <code>client_address[1]</code> (Port) từ tuple kết nối và trả về Client.
REQ-4 Tính Ổn định	Ngăn chặn gửi tin nhắn rỗng và đảm bảo chương trình không bị crash nếu một phía đóng kết nối.	Sử dụng vòng lặp kiểm tra <code>.strip() != ''</code> ở cả hai phía. Bao bọc các thao tác Socket bằng <code>try...except (ConnectionResetError, OSError)</code> .

Minh họa Trình tự Xử lý Giao thức Bài 1:

```
Client ---- [1] Connect TCP (port 5000) -----> Server
Client ---- [2] Send Username ('Nam\n') -----> Server (Server log: 'Nam từ 127.0.0.1: 54321 đã kết nối')
Client <--- [3] Welcome Message ----- Server
Client ---- [4] Send '/time\n' -----> Server (Server tự động trả lời thời gian hiện tại)
Client ---- [5] Send '/whoami\n' -----> Server (Server tự động trả lời IP và Source Port)
Client ---- [6] Send '/quit\n' -----> Server (Server phản hồi 'Tạm biệt!' và ngắt kết nối an toàn)
```

3. Bài 2: Chi tiết Triển khai File Transfer TCP

Bài 2 đòi hỏi truyền tải các tệp nhị phân thực tế (như ảnh PNG/JPG, tệp văn bản TXT, tài liệu PDF, tệp nén ZIP) qua cổng 5001 dưới dạng luồng byte (byte stream) nguyên bản.

Cấu trúc Quy chuẩn Giao thức 3 Bước (Application Protocol):

Bước	Đối tượng gửi	Nội dung Giao thức (UTF-8 / Binary Payload)
Bước 1	Client	Gửi một dòng Header dạng UTF-8: <filename> <size in bytes>\n Ví dụ: <code>document.pdf 2048500\n</code>
Bước 2	Client	Truyền đi đúng <size in bytes> byte dữ liệu nhị phân của tệp thông qua cơ chế đọc khối <code>BUFFER_SIZE = 64KB</code> .
Bước 3	Server	Server tiến hành lưu tệp vào thư mục <code>received_files/</code> và phản hồi lại Client dòng văn bản: • Thành công: <code>OK <filename> <size in bytes>\n</code> • Thất bại: <code>ERROR <mô tả lỗi>\n</code>

Cơ chế Dọn dẹp Tệp Lỗi dở dang (Partial File Cleanup):

Một điểm nâng cấp quan trọng trên Server là hàm `recv_exact_to_file()`. Nếu trong quá trình nhận tệp, Client bị ngắt kết nối mạng hoặc tắt ứng dụng đột ngột giữa chừng, hệ thống sẽ tự động bắt ngoại lệ và xóa bỏ tệp chưa hoàn chỉnh trong `received_files/` bằng lệnh `output_path.unlink(missing_ok=True)`, tránh tình trạng lưu trữ tệp rác bị hỏng trên đĩa cứng.

4. Kết quả Thử nghiệm & Kiểm thử Tự động

Dự án đã xây dựng bộ kịch bản kiểm thử tự động test_lab.py để kiểm tra toàn bộ luồng hoạt động của cả Bài 1 và Bài 2. Kết quả kiểm thử thành công 100%:

STT	Kịch bản Kiểm thử	Dữ liệu đầu vào / Lệnh	Kết quả Kỳ vọng & Thực tế	Trạng thái
1	Handshake Username	Username: NguyenVanA	Server in đúng log NguyenVanA từ 127.0.0.1:xxx đã kết nối và phản hồi chào mừng.	PASSED
2	Lệnh tự động /time	Tin nhắn: /time	Server tự động phản hồi chuỗi ngày giờ 2026-09-16 09:47:00 mà không cần gõ tay.	PASSED
3	Lệnh tự động /whoami	Tin nhắn: /whoami	Server tự động trả lại đúng IP 127.0.0.1 và Source Port ngẫu nhiên của Client.	PASSED
4	Gửi tin rỗng / Ngắt kết nối	Nhập phím Enter rỗng	Client thông báo ' <i>Không gửi tin nhắn rỗng</i> ' và yêu cầu nhập lại, chương trình không crash.	PASSED
5	Truyền tệp Nhị phân 128KB	Tệp test_image.png	Server lưu thành công tệp 131,072 bytes và trả về đúng chuỗi OK test_image.png 131072.	PASSED

5. Kết luận & Bài học Kinh nghiệm

Các bài học cốt lõi rút ra qua bài lab:

- **Định dạng Giao thức Ứng dụng (Application Protocol Design):** Việc quy định chính xác chuỗi kết thúc \n giúp việc tách dòng nhận dữ liệu văn bản trở nên đơn giản và chính xác.
- **Phân tách giữa Text Protocol và Binary Payload:** Việc đọc dòng Header bằng recv_line() UTF-8 kết hợp nhận exact bytes cho khối dữ liệu tệp giúp truyền chính xác mọi loại định dạng tệp mà không bị lỗi đệm mã hóa.
- **Xử lý Ngoại lệ mạng Robust:** Việc chủ động bắt các ngoại lệ ConnectionResetError và OSError giúp Server duy trì khả năng hoạt động liên tục (Always-on Availability) ngay cả khi Client bị ngắt kết nối đột ngột.

ĐẠI DIỆN NHÓM THỰC HIỆN DỰ ÁN
(Ký và ghi rõ họ tên)